MASTER table for Desmarestia dudresnayi

PUBLISHED
March 21, 2025

Preface

This notebook is aiming at setting the master table for good. Desmarestia dudresnayi annotation has 33 000 entries where the sister species herbacea has only ~15 000. Here I will go step by step on how I make the master table with all the features.

Ressources

Here I am listing the files I used for nf-core pipelines.
fasta: `/ebio/abt5_projects/reference_sequences/Desmarestia/Desmarestia_dudresnayi/genome/reference/Desmarestia-dudresnayi.chr.fa` **gff**: `/ebio/abt5_projects/PhaeoChromo/data/gtf-gff-bed-files/Desmarestia-dudresnayi_BAKER.chr.AGAT.gff`

Note: this gff file is derived from the reference gtf file, using AGAT tool to fix it and make it compatible with nf-core pipelines.

Code previously used to make the master table `./code/03_MASTER_TABLE_Desmarestia-dudresnayi.R` and then to add signatures to it `07_michael_mergeCS_v3.r`.

```
# Setting up environment
library(here)  # Optional but recommended for better path management
```

```
here() starts at /home/jeromine/Documents/Scripts_Rstudio/MASTER_TABLES/Notebooks
```

```
knitr::opts_knit$set(root.dir = "/home/jeromine/Documents/Scripts_Rstudio/MASTER_TABLE
knitr::opts_chunk$set(echo = TRUE)
```

Cleaning up GFF file

```
# Loading lib ------------------------------------------------------------
library(rtracklayer)
library(dplyr)
library(dlookr)
library(tidyr)
library(stringr)
library(data.table)
library(GenomicRanges)
library(ggplot2)
library(ggridges)
```

Step 1: Load GFF File

```
# Load GFF file
gff <- import("./data/gff_files/Desmarestia-dudresnayi_BRAKER.chr.gff_sorted", format
# Convert GRanges to a data frame
gff <- as.data.frame(gff)
# Convert to data.table for fast processing
setDT(gff)
print(sum(gff$type == "mRNA"))
```

```
[1] 33160
```

Step 2: Remove Duplicated Gene Model between AUGUSTUS and GeneMark

Screenshot in IGV that shows overlaping AUGUSTUS and GeneMark model:

Screenshot of IGV showing overlapping AUGUSTUS and GeneMark model

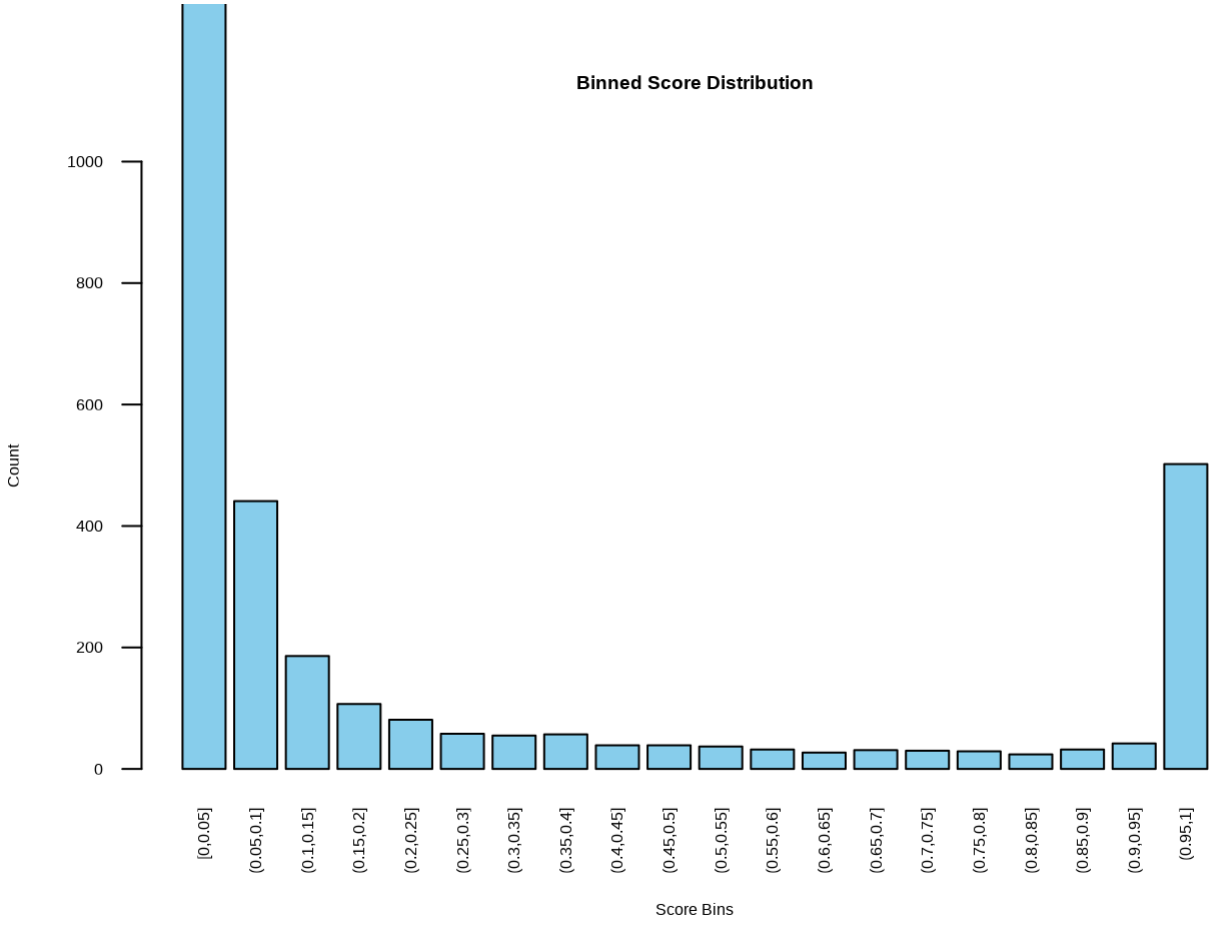
```
# Remove duplicated gene model
gff <- subset(gff, source == "AUGUSTUS")
print(sum(gff$type == "mRNA"))
```

```
[1] 31767
```

Option1 : multi-filtering

Step 3: Remove TE-Overlapping Genes

```
gff_data <- gff
CDS_TE_table <- read.table(file = './data/AGORE_TE-filtering_rcraig_for_jero/dDud.CDS-

# Define bins (adjust 'breaks' as needed for better granularity)
CDS_TE_table$score_bins <- cut(CDS_TE_table$score, breaks=seq(0, 1, by=0.05), include.

# Count occurrences in each bin
score_counts <- table(CDS_TE_table$score_bins)

# Create a barplot
barplot(score_counts, main="Binned Score Distribution", xlab="Score Bins", ylab="Count
```




```
# Filter overlap > 70%
CDS_TE_table <- CDS_TE_table %>% filter(score > 0.7)
CDS_TE_table$gene_id <- gsub('\\.t.*','',CDS_TE_table$trad_id)

# Filter out genes that are in the TE list
gff_data$GParent <- ifelse(is.na(gff_data$Parent),'',gsub('\\.t.*','',gff_data$Parent)
# Ensure Parent column does not contain 'character(0)'
gff_data$Parent <- ifelse(gff_data$Parent == "character(0)", NA, gff_data$Parent)
gff_data <- gff_data[!gff_data$GParent %chin% CDS_TE_table$gene_id, ]
gff_data <- gff_data[!gff_data$ID %chin% CDS_TE_table$gene_id, ]
print(sum(gff_data$type == "mRNA"))
```

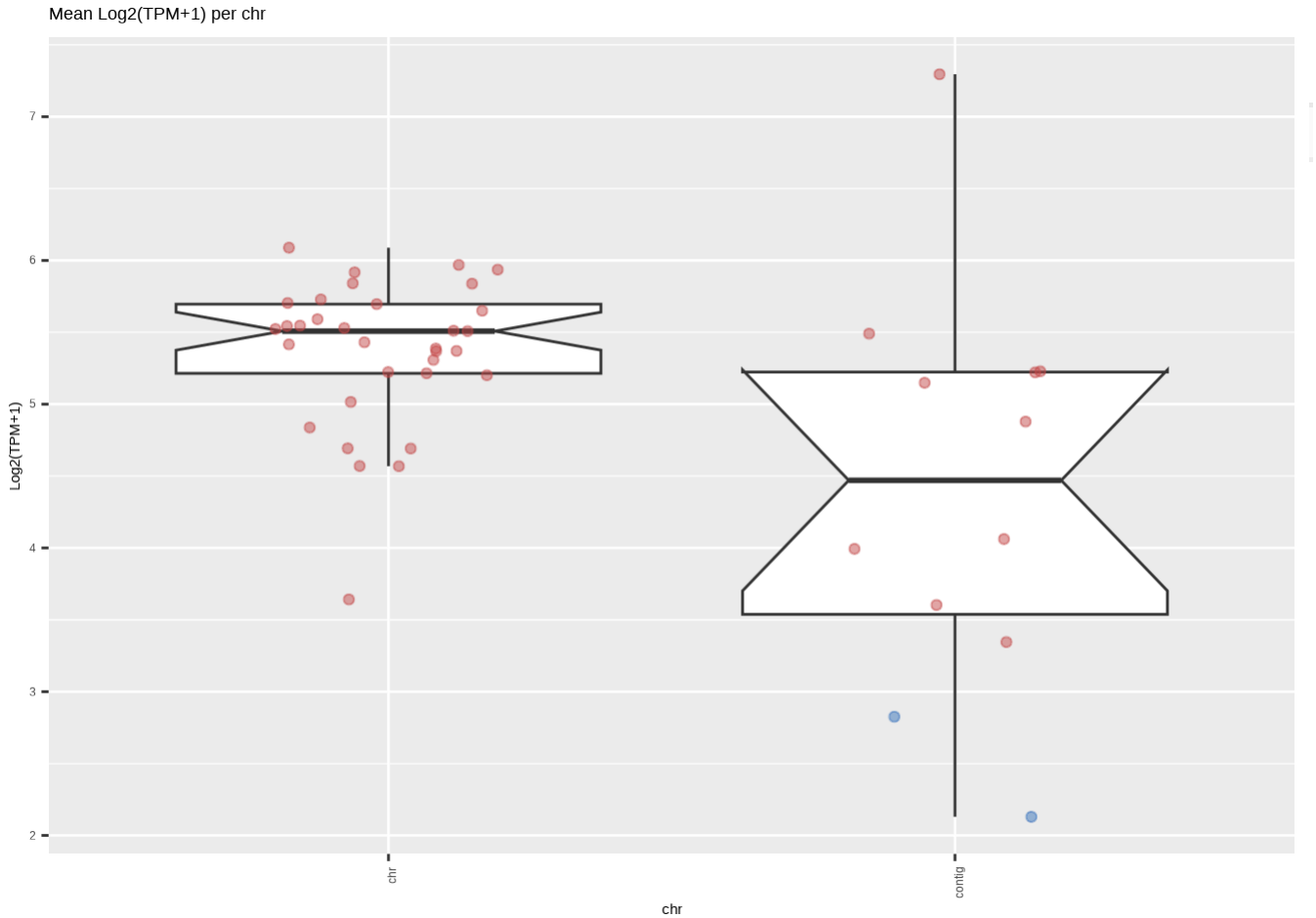
```
[1] 31103
```

Step 4: Filter Bacterial Contigs

```
## Define bacterial contigs: Bacterial contigs will be contigs where there are no read
# Load TPM data
tpm_data <- read.table('./data/my_RNAseq_data/Desmarestia-dudresnayi/salmon.merged.gen
# Calculate the TPM mean
tpm_data <- tpm_data %>%
  mutate(meanTPM = rowMeans(select(., Dd1, Dd2, Dd3), na.rm = TRUE)) %>%
  select(gene_id, meanTPM)
```

```
# Merge TPM with GFF data to get seqid
gff_tpm_data <- merge(gff_data, tpm_data, by.x = "GParent", by.y = "gene_id")

# Group by seqid, calculate mean TPM, and log-transform
tpm_summary <- gff_tpm_data %>%
  group_by(seqnames) %>%
  summarise(mean_scaffold_TPM = mean(meanTPM, na.rm = TRUE)) %>%
  mutate(log2_scaffoldTPM_plus_1 = log2(mean_scaffold_TPM + 1)) %>%
  mutate(cat = case_when(grepl("^chr", seqnames) ~ "chr",
                         grepl("^contig", seqnames) ~ "contig",
                         TRUE ~ "other"))

# Plot mean TPM per seqid
ggplot(tpm_summary, aes(x = cat, y = log2_scaffoldTPM_plus_1)) +
  geom_boxplot(outlier.shape = NA, notch = TRUE) +
  geom_jitter(aes(color = ifelse(cat == "contig" & log2_scaffoldTPM_plus_1 < 3, "#3F78
              width = 0.2, alpha = 0.5) +
  scale_color_identity() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  labs(title = "Mean Log2(TPM+1) per chr", x = "chr", y = "Log2(TPM+1)")
```

```
Notch went outside hinges
ℹ Do you want `notch = FALSE`?
```



```
# List bacterial contigs
bacterial_contigs <- tpm_summary$seqnames[tpm_summary$cat == 'contig' & tpm_summary$lo
print(length(bacterial_contigs))
```

```
[1] 2
```

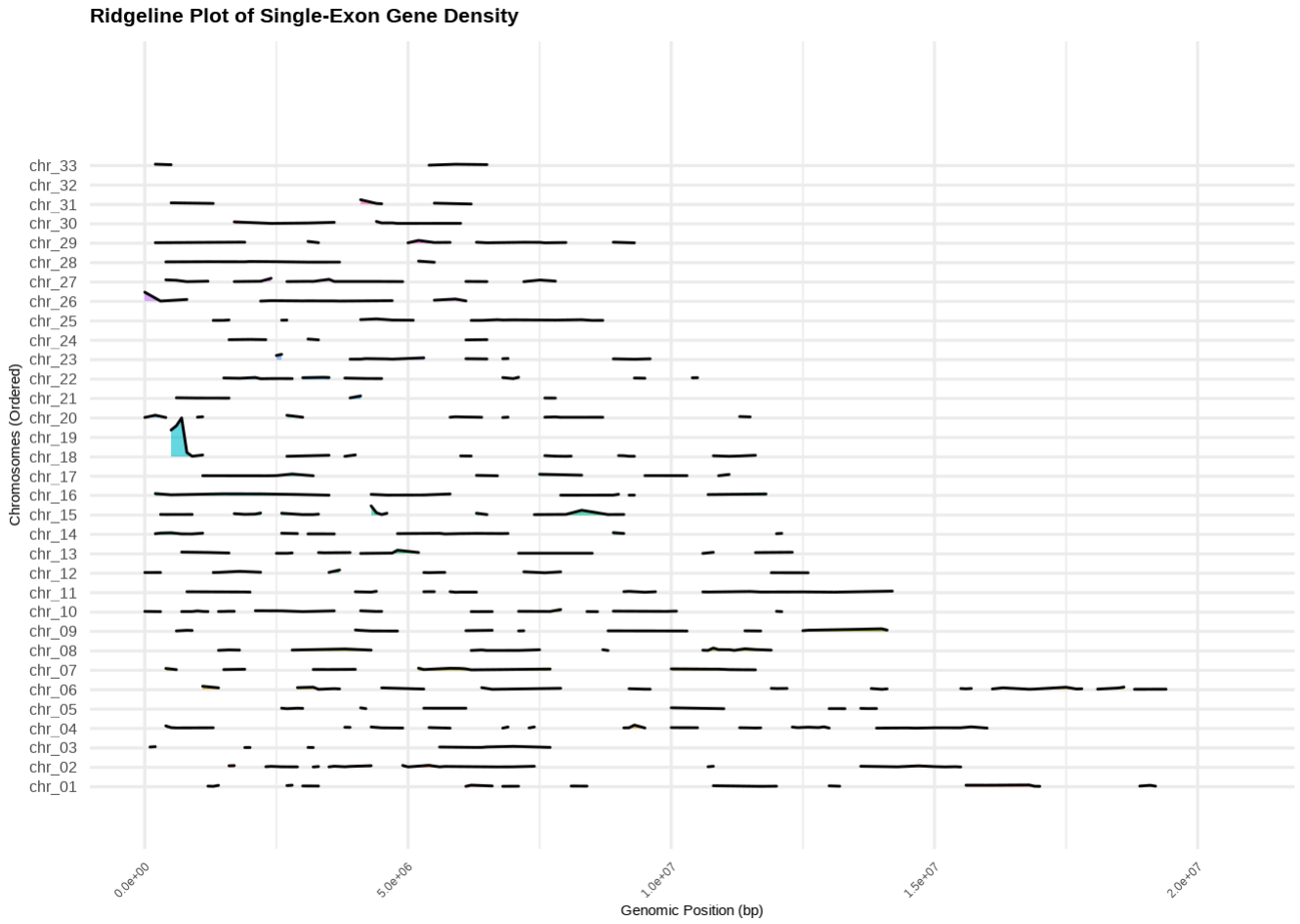
```
# Remove bacterial contigs
gff_data <- gff_data[!grepl(paste(bacterial_contigs, collapse = "|"), seqnames), ]
print(sum(gff_data$type == "mRNA"))
```

```
[1] 31097
```

Step 5: Remove Monoexonic Bacterial Genes

```
# Identify exon entries
exon_data <- gff_data[gff_data$type == "exon"]

# Count exons per gene using dplyr
gene_exon_counts <- exon_data %>%
  as.data.frame() %>%
  group_by(Parent) %>%
  summarise(exon_count = n(), .groups = "drop")
gene_exon_counts <- gene_exon_counts %>%
  mutate(GParent = gsub('\\.t.*','',gene_exon_counts$Parent))

# Shortlist monoexonic genes
single_exon_genes <- gene_exon_counts %>%
  filter(exon_count == 1)

# Filter genes with only one exon
single_exon_data <- exon_data[exon_data$GParent %chin% single_exon_genes$GParent]

# Define window size (e.g., 100,000 base pairs)
window_size <- 100000
# Compute monoexon density
single_exon_density <- single_exon_data %>%
  mutate(window = floor(start / window_size) * window_size) %>%  # Assign window bin
  group_by(seqnames, window) %>%
  summarise(SE_bp = sum(width), .groups = "drop") %>%  # Sum monoexon base-pairs per w
  mutate(SE_density = SE_bp / window_size)  # Compute density

# Ensure seqnames (chromosome names) are properly formatted
single_exon_density_toplot <- single_exon_density %>%
  filter(grepl("^chr", seqnames))  # Keep only seqnames that start with "chr"
write.csv(single_exon_density, './Notebooks/single-exon-density.csv', quote = FALSE, r

# Ridgeline plot with increased spacing and better scaling
ggplot(single_exon_density_toplot, aes(x = window, y = seqnames, height = SE_density,
  geom_density_ridges(stat = "identity", alpha = 0.6, scale = 2, rel_min_height = 0.01
  labs(title = "Ridgeline Plot of Single-Exon Gene Density",
```

```
       x = "Genomic Position (bp)",
       y = "Chromosomes (Ordered)") +
  scale_y_discrete(expand = expansion(mult = c(0.1, 0.2))) +  # Ensures spacing
  theme_minimal() +
  theme(
    axis.text.y = element_text(size = 12),
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 16, face = "bold"),
    legend.position = "none"
  ) +
  expand_limits(y = 0)  # Ensures taller rows don't get cut off
```

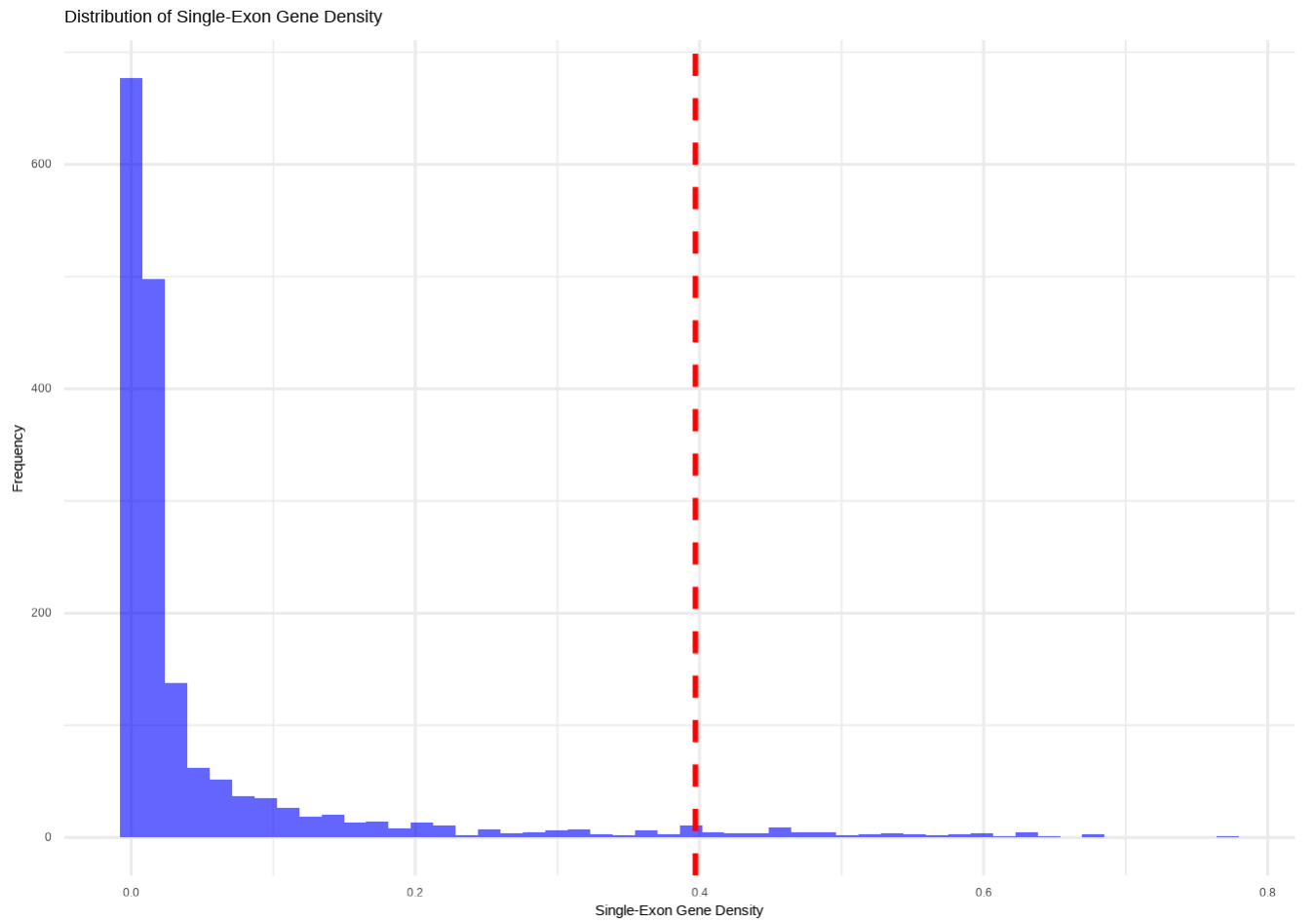



```
# Save a bigger version
ggsave("./Notebooks/monoexon_density_ridgeline_tall.png", width = 14, height = 15, dpi

# Compute the mean and standard deviation of SE_density
mean_density <- mean(single_exon_density$SE_density, na.rm = TRUE)
std_density <- sd(single_exon_density$SE_density, na.rm = TRUE)

# Define the Z-score threshold (mean + 3*std)
z_threshold <- mean_density + 3 * std_density

# Identify regions above the threshold
contaminated_regions <- single_exon_density[single_exon_density$SE_density > z_thresho
  select(seqnames, window, SE_bp)
```

```
# Convert to BED format (0-based start, end = window + SE_bp)
contaminated_regions <- contaminated_regions %>%
  mutate(start = window,
         end = window + SE_bp) %>%
  select(seqnames, start, end)

# Print the threshold
print(paste("Z-score threshold:", round(z_threshold, 5)))
```

```
[1] "Z-score threshold: 0.39715"
```

```
# Plot the distribution of SE_density with the threshold
ggplot(single_exon_density, aes(x = SE_density)) +
  geom_histogram(bins = 50, fill = "blue", alpha = 0.6) +
  geom_vline(xintercept = z_threshold, color = "red", linetype = "dashed", linewidth =
  labs(title = "Distribution of Single-Exon Gene Density",
       x = "Single-Exon Gene Density",
       y = "Frequency") +
  theme_minimal()
```




```
# Filter out monoexonic genes
# Convert to GRanges objects
gff_data <- gff_data %>%
```

```
  mutate(attributes = paste0("ID=", ID, ifelse(!is.na(Parent), paste0(";Parent=", Pare
  mutate(score = ifelse(is.na(score), '.', score)) %>%
  mutate(phase = ifelse(is.na(phase), '.', phase)) %>%
  select(seqnames,source,type,start,end,score,strand,phase,attributes)
gff_gr <- GRanges(seqnames = gff_data$seqnames, ranges = IRanges(start = gff_data$star
contaminated_gr <- GRanges(seqnames = contaminated_regions$seqnames, ranges = IRanges(

# Find overlaps
overlaps <- findOverlaps(gff_gr, contaminated_gr, type = "within")

# Remove contaminated features
clean_gff <- gff_data[-queryHits(overlaps), ]

# Save the masked GFF file
write.table(clean_gff, "./Notebooks/Dd_masked_output.gff", sep = "\t", quote = FALSE,
clean_gff <- as.data.table(clean_gff)

print(sum(clean_gff$type == "mRNA"))
```

```
[1] 28756
```

Option2 : Keeping orthologous genes from herbacea only

```
# Convert GRanges to a data frame
gff_data2 <- as.data.frame(gff)
# Convert to data.table for fast processing
setDT(gff_data2)
gff_data2$GParent <- ifelse(is.na(gff_data2$Parent),'',gsub('\\.t.*','',gff_data2$Pare

OG <- read.table('./data/Orthofinder/Results_Dec12/Orthogroups/Orthogroups.tsv', heade
rownames(OG) <- OG[,1]
OG <- OG %>%
  select(Desmarestia.dudresnayi_BRAKER_proteins, Desmarestia.herbacea_MALE_proteins) %
  filter(!is.na(Desmarestia.herbacea_MALE_proteins) & Desmarestia.herbacea_MALE_protei
OG <- OG %>%
  select(Desmarestia.dudresnayi_BRAKER_proteins) %>%
  separate_rows(Desmarestia.dudresnayi_BRAKER_proteins, sep = ",") %>%
  rename(trad_id = Desmarestia.dudresnayi_BRAKER_proteins)
OG$gene_id <- gsub('\\.t.*','',OG$trad_id)

gff_data2 <- gff_data2[gff_data2$GParent %chin% OG$gene_id, ]

gff_data2 <- gff_data2 %>%
  mutate(attributes = paste0("ID=", ID, ifelse(!is.na(Parent), paste0(";Parent=", Pare
  mutate(score = ifelse(is.na(score), '.', score)) %>%
  mutate(phase = ifelse(is.na(phase), '.', phase)) %>%
  select(seqnames,source,type,start,end,score,strand,phase,attributes)

print(sum(gff_data2$type == "mRNA"))
```

```
[1] 10641
```

Step 7: Export Cleaned GFF File

```
# Write new gff file

# Write few comments
cat('# This file was generated from reference gtf Desmarestia-dudresnayi, then convert
   # 1 - Removing duplicated gene model, 2 - Removing TE genes, 3 - Removing contigs p
file = "./data/gff_files/Desmarestia-dudresnayi.cleaned2025.jv.gff")
write.table(clean_gff, file = "./data/gff_files/Desmarestia-dudresnayi.cleaned2025.jv.

cat('# This file was generated from reference gtf Desmarestia-dudresnayi, then convert
   # 1 - Removing duplicated gene model, 2 - Keeping Desmarestia herbacea orthologous
file = "./data/gff_files/Desmarestia-dudresnayi.cleaned2025_OGonly.jv.gff")
write.table(gff_data2, file = "./data/gff_files/Desmarestia-dudresnayi.cleaned2025_OGo
```

MASTER TABLE

see next notebook